PicoBricks, explained from zero

Everything that happened in the AR part of our Creative Robotics project, from the first install to the ghost robot building a wall in the headset. Written so that someone who has never programmed, never used GitHub and never touched VR, AR or a robot can follow it.

TU Delft · MSc Architecture · ARIT1507 Creative Robotics in Spatial Design
Group: Matthias Nahum, Parjanya Singh, Suzanne Minkels, Jin Kim
Covers 9 – 21 October 2026 · Written for Jin Kim, to listen to (NotebookLM) and to explain to others

How to use this document. It is long on purpose: every term is explained the first time it appears, and the important ideas come back with an everyday comparison ("analogy" boxes in blue). Part 1 is the big picture. Part 2 is a dictionary of words. Part 3 tells the story day by day. Part 4 follows one wall through every device and every conversion. Part 5 lists every file type. Part 6 gives the problems we hit and what they teach. Part 7 is what is still open, plus ready-made explanations of 1 and 5 minutes.

Contents

1. The project in one page
2. The words you need (a beginner's dictionary)
3. The devices and programs, and what each one does
4. The story, day by day (9 – 21 October)
5. How one wall travels through the whole system (inputs, outputs, conversions)
6. Every file type in the project
7. Where everything lives (GitHub folder map, other places)
8. Problems we hit and what they teach
9. The rest of the team and the two robot routes
10. Open questions, next steps, and how to explain it yourself

1. The project in one page

The idea

The group's question is: *how can robots assist or enhance humans in design?* Instead of the robot being a labourer, it becomes a creative partner. Our concrete experience: a visitor puts on a headset, **designs a small wall** by drawing a curve on a real table, sees the wall appear in front of them in **augmented reality**, watches a **see-through "ghost" robot** build it in the headset, confirms, and then the **real UR5 robot** builds it with real timber bricks. Afterwards a camera-based system (Suzanne) lets the robot take the wall apart again, so the next visitor can start fresh.

Who does what

Person	Part
Jin Kim + Parjanya Singh	The AR app on the Pico headset (this document). Parjanya also wrote the first version of the laptop "bridge" to the robot.
Matthias Nahum	The robot side: making the UR5 move bricks. His route goes through Grasshopper (inside Rhino).
Suzanne Minkels	Computer vision: a Raspberry Pi camera reads markers on the table and bricks, so the robot can take the wall apart.

The lecturer's key requirement

The user must **see the robot build the wall in the headset and confirm, before the real robot moves**. The lecturer pointed us to HANDZONe, a TU Delft system that shows a live robot in VR. We took HANDZONe's 3D models of the UR5 and its table, and built our own preview around them.

Where we are on 21 October

- ✅ The headset shows the real room (passthrough) with virtual bricks, table and robot lined up on a real table.
- ✅ You place the table with two touches, draw a curve, choose 1–3 layers, and six bricks immediately form a wall along the curve (preview).
- ✅ Press **A** to confirm; the wall is sent over Wi-Fi to the laptop.
- ✅ The laptop works out every joint angle the robot needs (our own robot maths), and sends the movements back.
- ✅ The ghost robot in the headset picks each brick from the tray and builds the wall, exactly as the real robot would.
- ⏳ Not yet: running it on the real UR5 at the LAMA table (needs a lab session with Matthias).

The whole thing in one comparison. Think of an architect, a messenger and a builder. The headset is the **architect's sketchbook**: you draw, and it turns your sketch into an exact list of brick positions. The laptop is the **messenger plus the builder's foreman**: it receives the list and works out every move the builder's arm must make. The robot is the **builder**. Before the builder lifts anything, the foreman sends a **rehearsal** back to the sketchbook: a see-through robot acts out the build, so you can say "yes, go" or "no, let me redraw".

2. The words you need

Grouped by topic. You don't need to memorise these; come back when a word is unclear.

Reality: VR, AR and passthrough

Word	Meaning
VR (virtual reality)	You see only a computer-made world; the real room is hidden.
AR (augmented reality)	You see the real world with computer-made objects added on top.
MR (mixed reality)	Often used for AR where virtual things respond to the real room. Our app is AR/MR.
XR (extended reality)	Umbrella word for VR, AR and MR. Unity's tools for headsets are called "XR".
Passthrough (video see-through, VST)	The headset's outside cameras film the room and show it on the screens inside, live. That is how a VR headset becomes an AR headset. The Pico calls it "Video Seethrough".
Tracking	The headset constantly measures where it and the controllers are in the room (position and rotation), using its cameras and motion sensors.
Calibration	Telling the app where a real thing is. Our app doesn't know where your table is until you touch it with the controller.
Rig / XR Origin	The invisible "you" in the app: the camera (your eyes) plus the two controllers (your hands).
Ghost robot	Our see-through 3D model of the UR5 that acts out the build in the headset before the real robot moves.

Programming and apps

Word	Meaning
Code / script	Written instructions for the computer. Our app's scripts are in the language **C#** ("C sharp"); the laptop bridge is in **Python**.
Compile	Translate human-readable code into something the machine runs. "Compile error" = the translator found a mistake and stops.
Build	Package the whole project into one installable app.
APK	The installable app file for Android (the Pico runs Android). Like an .exe on Windows.
Sign / keystore	Android only installs apps that carry a digital signature. The signature comes from a private file, the **keystore** (`user.keystore`), protected by a password. It's like a wax seal: only the owner of the seal can make updates to the same app. Never upload it publicly.
SDK (software development kit)	A toolbox a company gives developers so their apps can use its hardware. The PICO SDK lets Unity talk to the Pico headset (passthrough, controllers, tracking).

Package	A ready-made add-on for Unity (for example the PICO SDK, or the XR Interaction Toolkit for grabbing things).
Batch mode	Running Unity without its window, from a command, to check that everything compiles or to run a setup step automatically.
Log / console	The running diary of a program: messages, warnings (yellow) and errors (red).
Bug	A mistake in a program that makes it behave wrongly. Debugging = finding and fixing it.

Unity words

Word	Meaning
Unity / Unity Editor	A "game engine": a program for building interactive 3D apps and games. The Editor is the window where you build the project.
Unity Hub	The launcher that installs Unity versions and opens projects. We use exactly version **6000.3.8f1**, the same as HANDZONe.
Project	The folder with everything for one app (our project folder is `PicoBricks`).
Scene	One "stage" in the app with everything on it (our scene: `SampleScene.unity`).
GameObject	Any thing in the scene: a brick, the table, the camera, an invisible helper.
Component	A piece of behaviour or property attached to a GameObject: its shape (mesh), its material, physics (Rigidbody), or one of our scripts.
Transform	Every GameObject's position, rotation and scale. Transforms can be nested: a child moves with its parent.
Prefab	A saved, reusable GameObject (like a template). The HANDZONe robot + table came in as a prefab.
Asset	Any file the project uses: models, materials, scripts, sounds.
Mesh / material	Mesh = the 3D shape (triangles). Material = how it looks (colour, transparency).
Rigidbody / kinematic	Rigidbody = the object obeys physics (gravity, collisions). Kinematic = physics switched off; the object stays exactly where the code puts it.
Collider	An invisible shape used for bumping into things.
URP	Universal Render Pipeline: Unity's way of drawing images, tuned for mobile devices like the Pico.
HDR	High dynamic range colour. In our case it caused a black background, because its colour format had no transparency (see Part 8).
Play mode	Running the app inside the Editor on the laptop, without the headset.
Build and Run (Ctrl+B)	Build the APK and install it on the connected headset.

Sharing work: Git and GitHub

Word	Meaning
Git	A program that keeps the full history of a project folder, so you can go back to any earlier version and work in parallel safely.
GitHub	A website that stores Git projects online, so a team can share them. Ours: `github.com/jintecture/PicoBricks` (private).
Repository (repo)	One project tracked by Git, with all its history.
Commit	A saved snapshot of the project with a short message ("what changed and why"). Like a page in a diary.
Push / pull	Push = upload your new commits to GitHub. Pull = download your teammates' commits.
Clone	Download a full copy of a repository for the first time.
Branch	A parallel line of work. You can try something on a branch without disturbing the main version (`main`).
Merge	Combine a branch back into another. A **merge conflict** happens when two people changed the same lines; a human picks the right version.
Pull request (PR)	On GitHub, a request "please merge my branch", where others can look first.
Tag	A permanent bookmark on one commit, like `v1-manual-placement`.
.gitignore	A list of files Git should not track (huge generated folders, the private keystore, passwords).
Git LFS	Large File Storage: an add-on for big files (HANDZONe uses it, 6 GB).

Git in one comparison. Git is a Google Docs version history for a whole folder, but with deliberate "save points" (commits) that carry a note, and with the ability to make a parallel copy (branch) to try an idea and later fold it back in (merge).

Robots

Word	Meaning
UR5	A robot arm by Universal Robots. 6 rotating joints, reach 850 mm, carries up to 5 kg. Ours is an older "CB3" model.
Joint	A motor that rotates one part of the arm. The six joints in order: **base** (turns the whole arm), **shoulder**, **elbow**, **wrist 1, wrist 2, wrist 3**.
Joint angles	Six numbers (degrees or radians) that fully describe the arm's pose, for example (0, −90, −90, −90, 90, 0).
Flange	The round metal plate at the end of the arm, where a tool is bolted on.

Tool / gripper / TCP	The gripper is the "hand" bolted to the flange. TCP (tool centre point) is the exact point of the tool that should go to a target, for example between the gripper's fingertips.
Forward kinematics (FK)	From joint angles to "where is the hand?". Easy: just follow the arm link by link.
Inverse kinematics (IK)	From "I want the hand here, pointing down" to "which joint angles?". Harder; for the UR5 there is a known exact formula with up to 8 answers (elbow up/down, wrist flipped, etc.).
Reach	How far from its base the robot can put its tool. 850 mm is the maximum; with the gripper pointing down it's less.
Teach pendant / Polyscope	The robot's tablet with buttons, and its software. You can read coordinates there and move ("jog") the arm by hand.
URScript	The UR robot's own little programming language (for example `movej` = move by joints, `movel` = move in a straight line).
MoveJ / MoveL	MoveJ: each joint turns smoothly to its new angle (the hand moves in a curve). MoveL: the hand moves in a straight line (used for going straight down to a brick).
RoboDK	Software that simulates robots on the laptop and can drive real ones. Its trial licence on Jin's laptop expired on 19 October.
URSim	Universal Robots' simulator of the robot's controller.
Digital twin	A virtual copy of a real machine that mirrors it. Our ghost robot is a preview twin.

Geometry and coordinates

Word	Meaning
Coordinate frame	A starting point plus three directions (x, y, z) you measure from. The same point has different numbers in different frames, like an address given "from the station" or "from the church".
Table frame (our "Workspace")	Our main frame. Origin = the middle of the table's front edge (where you stand), on the table top. x = to your right, "away" = into the table, and up.
Robot base frame	The robot's own frame, starting at the centre of its base, in millimetres.
y-up / z-up	Unity calls "up" y; robots, Rhino and most engineers call "up" z. Converting means swapping axes.
Left-handed / right-handed	Two mirror-image conventions for which way the third axis points. Unity is left-handed; robots are right-handed. This flips the sign of rotations.
Yaw	Rotation around the vertical axis ("which way does the brick point, seen from above").
Degrees / radians	Two units for angles: 180° = π radians ≈ 3.14. Robots often use radians.
Bounding box	The smallest box around an object. We measured the LAMA table model this way.

Networks

Word	Meaning
IP address	A device's address on a network, for example 192.168.0.150 (headset at home), 192.168.0.249 (laptop at home).
Port	A numbered "door" on a device, so different programs can receive their own messages. We use port **5005** (headset → laptop) and **5006** (laptop → headset).
UDP	A way of sending short messages without a handshake, like posting a postcard. Fast, but a postcard can get lost.
TCP	A way of sending with a handshake and delivery check, like a phone call. (Matthias's Grasshopper link uses TCP.)
Broadcast	Sending a message to every device on the local network at once, in case the exact address is wrong.
JSON	A plain-text format for structured data, with labels and values: `{"x": 0.12, "yaw": -90}`. Readable by people and by every programming language.
Hotspot	The laptop makes its own Wi-Fi network. Windows gives the laptop the address 192.168.137.1 there. At home we don't need it.
adb	Android Debug Bridge: a command-line tool to talk to an Android device over USB (install apps, read logs, copy files). Unity uses it to install our app on the Pico.
USB debugging / developer mode	Headset settings that allow a computer to install and debug apps over the cable.

3. The devices and programs

Hardware

Thing	What it is	Its job in the project
Pico 4 Enterprise (first headset)	A standalone VR headset (an Android computer on your head). Firmware (its operating system version) 5.9.9, Android 10. Borrowed.	Used 9–15 Oct. Its old firmware forced us to use an older PICO SDK (2.5.0).
Pico 4 Ultra Enterprise (current headset)	Newer model (A9210), firmware 5.15.5, Android 14. Managed by TU Delft (their apps are installed; we only ever install or remove PicoBricks).	Runs our app from 15 Oct, with PICO SDK 3.4.0.
Controllers	Two handheld trackers with buttons: trigger, grip, thumbstick, A/B (right), X/Y (left).	Calibrate, draw, choose layers, confirm.
Laptop (Windows 11)	Jin's computer.	Builds the app (Unity), runs the bridge (Python), stores the project (Git), talks to the headset over USB and Wi-Fi.
UR5 robot (CB3)	The robot arm in the LAMA lab, bolted to the LAMA table; later moved to the exhibition space.	Builds the real wall.
LAMA table	The table the robot is mounted on: 143 × 61 cm, 91 cm high (measured from the HANDZONe 3D model).	The real table the AR scene is lined up with.
Timber bricks	5 × 5 × 10 cm, 0.12 kg each, 6 of them.	What the robot builds with.
Pickup tray	A 3D-printed holder (PLA plastic, Bambu A1 printer) with 6 pockets, 229 × 250 × 19.8 mm.	Makes sure each real brick starts in a known, repeatable spot for the robot to pick up.
Raspberry Pi + camera (Suzanne)	A tiny computer with a camera, on a tripod.	Reads markers to see the bricks, for taking the wall apart.

Software

Program	Where it runs	Its job
Unity 6000.3.8f1 + Unity Hub	Laptop	Building the headset app.
PICO Integration SDK (2.5.0, then 3.4.0)	Inside the Unity project / the app	Lets the app use the Pico's passthrough, tracking and controllers.
XR Interaction Toolkit 3.4.0 (+ "Starter Assets")	Inside the Unity project	Ready-made controller laser, grabbing, the XR rig.

Our app "PicoBricks" (`com.lama.picobricks`)	Headset	Everything the user sees and does.
Python 3.10 (comes with RoboDK)	Laptop	Runs the bridge program.
The bridge: `picobricks_bridge.py` + `PICOBRICKS.bat`	Laptop	Receives walls from the headset, converts them for the robot, works out the robot's movements, sends the preview back, later drives the robot.
`ur5_kinematics.py`	Laptop (used by the bridge)	Our own UR5 joint maths (replaces RoboDK for the preview).
RoboDK	Laptop	Robot simulation. Used on 19 Oct; trial expired, now optional.
Rhino 8 + Python script	Laptop	Generates the pickup tray's 3D model automatically (`make_tray.py`).
Bambu Studio	Laptop	Prepares the tray's STL file for the 3D printer ("slicing").
Git + GitHub	Laptop + website	Version history and sharing with the team.
HANDZONe (lecturer's)	Laptop (local copy only)	Source of the UR5, gripper and LAMA table 3D models. We never changed or pushed anything there.
Grasshopper + Robots plugin (Matthias, Suzanne)	Laptop (Rhino)	The other robot route; also used by the computer vision part.
adb	Laptop	Installs the app on the headset, copies recordings off it.
Obsidian vault + Google Drive	Laptop / cloud	Jin's notes and copies of headset screenshots and recordings.



USB cable (adb): the laptop installs the app on the headset and copies screenshots/recordings off it.

Wi-Fi, UDP port 5005: headset → laptop, the confirmed wall (JSON).

Wi-Fi, UDP port 5006: laptop → headset, ~200 robot poses for the ghost (JSON).

Ethernet (dashed, not used yet): laptop → robot, URScript on port 30002 (or via RoboDK), robot IP 192.168.1.198.

Internet: laptop ↔ GitHub (code and history), laptop ↔ Google Drive (notes, recordings).

4. The story, day by day

Friday 9 October: getting the tools working

- **Unity Hub and Unity 6000.3.8f1.** We installed exactly the version HANDZONe uses, because opening a project in another version upgrades it and can break it. The first install was missing the **Android modules** (needed because the Pico is an Android device). Fix: remove and reinstall with Android Build Support, OpenJDK, Android SDK and NDK.
- **HANDZONe** (the lecturer's project on GitHub) was cloned (6 GB, with Git LFS) and opened, to look at it. It is TU Delft New Media Centre's system to control UR robots from VR: a Unity app with a live robot twin, a server in Docker, and the real robot or URSim. It targets the Meta Quest, so we decided to use it as a **source of parts**, not as our app.
- **Decision: a fresh small project, "PicoBricks".** Debugging a new headset inside a big unfamiliar project is hard; small is easier to understand.
- **Headset setup:** developer mode and USB debugging on, connected with the PICO cable (a charge-only cable didn't work).
- **PICO SDK** added; "Apply All" and "Fix All" set the right Android settings (Android API 29, IL2CPP/ARM64, the PICO XR plug-in).
- **Keystore** `user.keystore` created so the app can be signed. Unity forgets its password on every restart, so you re-enter it before building. It is never uploaded to GitHub.
- ✅ First build: a grey cube floating in the headset.

Saturday 10 October: grabbing bricks

- Wrote `BrickSceneSetup.cs`: a menu item **PicoBricks → Setup Brick Scene** that builds the whole scene in one click (rig, table, floor, bricks with physics and grabbing). This way every teammate gets an identical scene, and nobody sets up 50 things by hand.
- ✅ Picking up and stacking bricks in VR worked. Reset button (X) added.
- Passthrough attempt: the background stayed **black**, and the X/Y buttons did nothing. The buttons were fixed by reading the controllers the same way the PICO SDK does (`UnityEngine.XR.InputDevices`).

Sunday 11 October: seeing the real room, and placing the table in it

- **Black passthrough, cause 1:** the old headset's firmware (5.9.9) was too old for PICO SDK 3.x. We don't update a borrowed headset's firmware, so we **downgraded the SDK to 2.5.0**.
- **Black passthrough, cause 2:** URP's HDR colour format has no transparency ("alpha") channel, so the "see-through" background was painted solid black. Fix: HDR off while passthrough is on. ✅ The real room became visible.
- **Table floating above the head:** on that firmware, switching passthrough on shifted the tracking height by ~1.15 m. Decision: **don't trust the headset's floor; measure the real table by touching it**.

- **Table calibration** (`TableCalibration.cs`): press B, touch points with a small white ball in front of the controller, pull the trigger. Version 2 used 4 points: front-left corner, front-right corner, back-left corner, floor. The virtual table copies the real width, depth, height and direction.
- Bugs fixed: bricks falling forever (the invisible floor was in the wrong place → it now moves with the table) and the table dropping 1 m with every calibration (the ready-made rig had **gravity and walking** built in, meant for games → all switched off; in AR the world must stay locked to the room).
- Git set up; first commit. Decisions written down: a 3D-printed tray for repeatable brick positions; measuring table marks with the robot to link headset and robot coordinates.

Tuesday 13 October: grid snap (Parjanya)

- Parjanya made `BrickSnap.cs`: a released brick turns to the nearest 90°, snaps to a 5 cm grid or centres on the brick below, and locks in place. Version 2 added standing bricks and stopped bricks floating when one below was removed.
- His laptop had trouble installing on the headset (adb couldn't find it); workaround: install the APK by hand.

Wednesday 14 October: the pickup tray

- **Problem:** the robot needs each brick to start in exactly the same place. **Solution:** a 3D-printed tray.
- The tray is **generated by a Python script inside Rhino 8** (`make_tray.py`) instead of modelled by hand, so a measurement can be changed and the tray regenerated. Output: STL files for the printer.
- Rhino's boolean operations (cutting shapes out) failed at first: some cutting shapes were inside-out. Fix: turn them outward, merge all cutters, subtract once.
- From 10 bricks in two trays to **6 bricks in one tray**. Clearance tested with small printed test frames: **0.5 mm per side** fits the real bricks.
- Later redesign: corner pieces with big 45° slopes that meet in a ridge, so a brick landing anywhere near a pocket slides into it (229 × 250 × 19.8 mm).

Thursday 15 October: new headset, new idea

- Parjanya's grid snap merged into `main` (Pull Request #1). The state was bookmarked as tag `v1-manual-placement`.
- **New headset:** Pico 4 Ultra Enterprise, firmware 5.15.5. It needs SDK 3.x and its firmware is new enough, so we went **back to PICO SDK 3.4.0**.
- **Version 2, the big change of direction:** instead of placing bricks one by one, you **describe the wall**: draw a line, choose the number of layers (1–3), and the app works out every brick. With 6 bricks: 1 layer = 6 × 1 (60 cm), 2 layers = 3 × 2 (30 cm), 3 layers = 2 × 3 (20 cm). Grid snap switched off.
- After the first test, Jin's feedback: it must be a **curve**; bricks must touch at one corner and **mimic the curve**; bricks should start in the tray; bricks should be readable (brown, 70 % see-through, black edges). All built the same day (`WallBuilder.cs`), and the tray's STL was imported into Unity (`TrayMeshImporter.cs`, because Unity can't read STL by itself).
- ✅ Tested in the headset: "this is good".

Friday 16 – Sunday 18 October: the team's work comes together

- **Matthias** added a CSV exporter (brick positions for Rhino), merged via Pull Requests #2 and #3, and worked on a live Unity → Grasshopper link (branch `tcp-receiver`).
- **Parjanya** built the robot link (branch `robot-link`): `BrickSender.cs` in the app sends the bricks over Wi-Fi, and the laptop bridge converts them for the robot and drives RoboDK or the UR5.
- **Jin found a flaw in the curve logic** on an S-curve drawing: bricks drifted off the curve. Fixed by aiming every brick from where it really starts ("dividers" and "folding ruler", Part 5).
- **Everything merged into `main`**, carefully: bookmark tag `before-robot-link` first, then merges. The only conflicts were invisible line-ending differences. Parjanya's script didn't compile (two Unity toolkits used the same names); fixed by naming them in full.

Monday 19 October: headset → laptop → robot simulation, and the ghost robot

- RoboDK installed. ✅ **Headset → laptop worked** over home Wi-Fi.
- RoboDK errors fixed: "Invalid item" (RoboDK had been restarted → the bridge now reconnects), "Target cannot be reached" (the placeholder table was too far from the robot). ✅ RoboDK built the headset's wall. Big arrows replaced by small green footprints. RoboDK lagging behind fixed (only real duplicates skipped). Jin kept the **"Run? [Y/n]"** question on purpose: it gives time to take the headset off.
- **HANDZONe models imported**: UR5 + gripper + LAMA table (models only; a temporary script in our local copy of HANDZONe exported them and was deleted again).
- **Ghost robot placed**: Jin confirmed the real setup (LAMA table, user at the long side, robot on the right). The scene measures the table model itself. The robot is pale blue and see-through; the LAMA table only shows in VR mode.
- **Joint angle mismatch found and fixed:** HANDZONe's model stands upright at "all joints 0", the real UR5 lies flat. Shoulder and wrist 1 get +90°.
- Windows **Smart App Control** suddenly blocked one of Unity's own files ("An Application Control policy has blocked this file"). Jin turned it off in Windows Security.
- Tray moved to the back corner beside the robot; a black drawing area added.
- **RoboDK's licence had expired.** Decision (Jin): write our own UR5 maths (`ur5_kinematics.py`) instead of buying a licence. Tested: 2000 random poses, 0.000 mm error.
- ✅ **The ghost robot built the wall in the headset on the first try.**

Tuesday 20 October: making it usable

- **Bug:** "the robot can't reach" every time, tray on top of the robot. The robot was placed from the middle of the front edge, the tray from the left corner; on a narrower table the tray slid into the robot. Fixed: both measured from the same point.
- **Jin's idea: 2-point calibration** with the real LAMA table size, anchored at the front-right corner next to the robot, so testing on a different table at home still puts robot, tray and drawing area exactly where they'll be in the lab.

- Drawing area made bigger using the robot maths (the robot really reaches 11–81 cm from its base with the gripper pointing down; walls may go 20–75 cm). Walls that poke out now slide to the nearest fitting spot.
- Team message drafted; questions answered (Quest vs Pico: about a day of work, mainly passthrough).

Wednesday 21 October: catch-up slides, and the confirm step

- 6 slides made for the group's catch-up presentation, in the group deck's style, plus 4 diagrams of the curve logic; headset recordings copied to Google Drive.
- **Headset test findings:** 1-layer walls still refused (the old small drawing area was still stored in the scene → now cleared), bricks overlapping on S-curves (→ every brick is now checked against all earlier bricks), bricks floating and tilted (→ the A button was read by two scripts; pressing it during the ghost's build sent half-moved bricks as a new wall).
- **Jin's request: confirm before the robot builds.** Now: draw → the bricks immediately preview the wall → change layers with the thumbstick → **A = "yes, build it"** → only then it goes to the robot. ✅ Works.

5. How one wall travels through the whole system

This part follows one wall from the laptop where the app is made, into the headset, over Wi-Fi to the laptop, back to the headset, and (soon) to the robot. At every step: **what goes in, what comes out, and what is converted**.

Step 0. Making the app (laptop → headset, by USB)

In	What happens	Out
Unity project: C# scripts (`.cs`), models (`.fbx`, `.asset`), materials (`.mat`), the scene (`.unity`), settings	**PicoBricks → Setup Brick Scene** builds the scene: the XR rig, the table, floor, tray and 6 bricks, the ghost robot, and wires all the scripts together. Then **Ctrl+B**: Unity compiles the C# code into machine code for the headset's ARM processor (IL2CPP), packs everything, and signs it with the keystore.	An app file `PicoBricks.apk` (~60 MB)
`PicoBricks.apk`	Unity uses **adb** over the USB cable to install it on the headset and start it.	The app running on the headset

Step 1. Placing the table (inside the headset)

Why: when the app starts, it has no idea where your real table is. Calibration tells it.

1. Press **B**. A white ball sits 6 cm in front of the right controller: the "measuring tip".
2. Touch the table's **front-right corner** (next to the robot), pull the trigger. The app records that 3D point.
3. Touch **any point further left on the front edge**, pull the trigger.

Conversion: the two points give a direction ("right" along the front edge). "Away from you" is worked out as the direction at a right angle to it, flat on the table. The app knows the LAMA table's real size (143 × 61 cm, 91 cm high), so from the right corner it places the whole table: its front-edge middle becomes the **origin of the table frame** (the "Workspace"). Everything else (the robot, the tray, the bricks, the drawing area) is a child of this Workspace, so it all moves into place together.

> **Analogy.** Pinning a paper map onto a real field. Two pins are enough: the first fixes where the map is, the second fixes which way it's turned. Once pinned, everything printed on the map (robot, tray, wall area) lands in the right spot in the field.

(Without the ghost robot, an older 4-point version measures any table: front-left, front-right, back-left corner and the floor.)

Step 2. Drawing the curve (inside the headset)

- **Input:** you point the right controller's laser at the table and hold the trigger.

- The app finds where the laser ray crosses the table plane ("ray–plane intersection"), converts that point into table coordinates, keeps it inside the black outline, and stores a point every ~1 cm.
- When you release, the curve is **smoothed**: each point is averaged with its neighbours a few times, to remove hand wobble (the first and last point stay).
- **Output:** a list of points in table coordinates (metres), shown as a cyan line.

Step 3. From a curve to a wall: the brick logic (inside the headset)

1. **Resize.** One layer of bricks has a fixed length: 6, 3 or 2 bricks × 10 cm. The curve is scaled around its middle until it is exactly that long. A quick small sketch and a big sweep give the same shape at wall size.
2. **Dividers.** From where a brick starts, the app looks for the point on the curve exactly 10 cm away (a circle of 10 cm around the start, crossing the curve). The brick points there. The next brick starts where this one ends.
3. **Folding ruler.** Two straight bricks can't both follow a bend: one side opens a gap, the other would overlap. So neighbouring bricks share the corner on the **inside** of the bend, like the hinge of a folding ruler. Since 21 October the app tries both possible hinge corners for every brick, checks the candidate against **all** bricks laid so far (an exact overlap test, the "separating axis test"), and keeps the one that lands closest to the curve. If a bend is too sharp for 10 cm bricks, it is straightened just enough.
4. **Stack.** Every layer repeats the same chain, straight up (layer height 5 cm).
5. **Fit into the robot's area.** Every corner, edge midpoint and centre of the bottom bricks must be on the table, off the tray, and 20–75 cm from the robot's base. If not, the app tries sliding the whole wall (same shape) up to 40 cm in 1 cm steps and takes the smallest move that fits.

Output: a plan: for each brick, its centre (x, y, z in metres in the table frame), its rotation, and its footprint. The bricks immediately stand there as a **preview**. The thumbstick changes 1/2/3 layers and the preview updates. Trigger = draw again.

Step 4. Confirm, and send to the laptop (headset → laptop, Wi-Fi, port 5005)

Press **A** = "I'm happy with this wall". Then:

1. `WallBuilder` saves the plan on the headset as `wall_plan.json` and announces "confirmed".
2. `BrickSender` builds a message and sends it twice: straight to the laptop's address, and as a broadcast (in case the address is wrong). It's one UDP packet of about 1.5 KB.

Conversion inside BrickSender (Unity → robot-style axes): Unity uses x right, y up, z away, and measures yaw clockwise. The message uses x right, **y away, z up**, and yaw **counter-clockwise**, like robots and Rhino. So y and z are swapped and the yaw's sign is flipped. Yaw 0 means the brick's long side points away from you.

```
{ "type": "layout", "seq": 3, "frame": "table", "units": "m_deg",
  "bricks": [
    { "id": 1, "name": "Brick_1", "standing": false,
      "pick":  { "x": 0.1155, "y": 0.3915, "z": 0.025, "yaw": 0.0 },     ← its pocket in the tray
      "place": { "x": -0.150, "y": 0.120,  "z": 0.025, "yaw": -90.0 } },  ← its spot in the wall
    ...5 more... ],
  "robot": { "valid": true, "x": 0.47, "y": 0.31, "z": 0.0,             ← where the ghost robot
stands
             "armYaw0": 90, "armYaw90": 180, "check": [ ... ] } }        ← for the automatic line-
up check
```

`seq` is the wall's number, so the laptop can ignore the duplicate copy. The bricks are sorted bottom layer first, then along the curve: that's the build order.

Step 5. The laptop bridge converts the wall for the robot

The bridge (`picobricks_bridge.py`, started with `PICOBRICKS.bat` → option 3) waits on port 5005. When a wall arrives, it saves it as `last_layout.json` and converts it in several stages:

Stage	From → to	How
a. Line up with the ghost (simulation)	Headset's ghost position → a "table frame" for the robot	The headset says where the ghost robot stands and which way its arm points. The bridge compares that with its own UR5 maths, and works out how the robot is turned on the table. (For the real robot, `calib.json` is used instead: the robot's own coordinates of the table's two front corners, read off the teach pendant.)
b. Clean up	Brick positions → tidy positions	Heights are rounded to whole brick layers (the headset can be a few mm off and stacking isn't forgiving). A brick turned 180° looks the same, so the yaw is kept between −90° and 90°.
c. Table → robot base	metres in the table frame → millimetres from the robot's base	Move (subtract the base position), rotate (the robot's turn on the table), scale ×1000.
d. Brick centre → gripper target	where the brick is → where the gripper's tip must go	The tip goes to 25 mm below the brick's top face (grasp depth). For placing, 2 mm higher (so it releases just above). The tool points straight down, turned with the brick's yaw.
e. The route	pick and place targets → a sequence of moves	For each brick: go to 250 mm above the table over the pocket → straight down (last 80 mm slowly) → close gripper → straight up → over to the wall → straight down → open → up. Then back home.
f. Gripper tip → flange	tool tip → the end plate of the arm	The gripper is assumed 150 mm long, so the flange is 150 mm above the tip.
g. Inverse kinematics	flange position → 6 joint angles	`ur5_kinematics.py` solves it with the exact UR5 formula. Each pose can have up to 8 solutions (elbow up/down, wrist flipped…); it picks the one closest to the previous pose, so the arm moves smoothly instead of flipping. Straight moves are split into 4 in-between poses each.

Output: about 200 poses for 6 bricks, each a list of 6 joint angles; some poses are marked "close gripper on Brick_3" or "open". If anything can't be reached, an error message instead ("can't reach Brick_3 in the tray, 9 cm from the robot: too close to its base").

> **Inverse kinematics, in plain words.** When you reach for a cup, your brain chooses shoulder, elbow and wrist angles without you thinking about it. Inverse kinematics is that calculation for the robot: "hand there, pointing down" in, six joint angles out. Forward kinematics is the reverse and much easier: from the angles, follow the arm piece by piece to see where the hand ends up. We use forward kinematics to **check** the inverse: 2000 random poses, 0.000 mm error.

Step 6. The preview goes back to the headset (laptop → headset, port 5006)

```
{ "type": "preview", "seq": 3,
  "frames": [
    { "j": [0, -90, -90, -90, 90, 0] },                                ← one pose: 6 joint angles
(degrees)
    { "j": [-199.03, -68.2, -112.01, -89.78, 90.0, 70.97] },
    { "j": [-199.2, -75.8, -121.6, -72.5, 90.0, 70.8], "grip": "close", "brick": "Brick_1" },
    ...
    { "j": [-148.56, -97.69, -86.41, -85.9, 90.0, 211.44], "grip": "open", "brick": "Brick_1" },
    ...about 200 in total... ] }
```

About 12 KB, still one UDP packet. Sent to the address the wall came from.

Step 7. The ghost robot acts it out (inside the headset)

1. `GhostPlayer` receives the poses, remembers each brick's wall spot, and puts the bricks back in the tray.
2. It moves the ghost's joints smoothly from pose to pose (each joint turns at most 90° per second; this is "interpolation": filling in the in-between angles), like a flip-book.
3. At "close": the brick is attached to the robot's hand and travels with it. At "open": it's released exactly on its wall spot.
4. A label above the robot shows progress ("picking Brick_3 (3/6)"), then "Robot preview done. On the laptop: Run? [Y/n]".

Conversion here: real joint angles → the 3D model's rotations. HANDZONe's robot model is built from nested parts, each turning around its own local axis. Each joint's rotation = its starting rotation + direction (±1) × angle. Two extra corrections: the model's "zero" is upright while the real robot's zero lies flat, so the shoulder and wrist 1 get +90°.

Automatic check: the headset also sends where the ghost's hand is in 3 test poses; the bridge compares with its maths and prints "ghost robot matches the UR5 maths (within 35 mm)". The small difference is because the two measure slightly different points of the hand; a wrong joint direction would give tens of centimetres.

Step 8. The real robot (next step, not yet tested)

- The laptop asks **"Run? [Y/n]"** (a second safety confirmation, and time to take the headset off).
- Option 8 in the menu writes a **URScript** program and sends it to the robot (IP 192.168.1.198, port 30002) over an Ethernet cable. Option 7 uses RoboDK (needs a licence).
- Before that, at the robot: **menu 5** = jog the robot's tip onto the table's two front corners and type the coordinates (this creates `calib.json`); **menu 6** = the robot hovers 30 mm above both corners to check.
- Planned improvement: send the robot the **same joint angles** the ghost played, so it can't take a different route.

All the conversions at a glance

#	From	To	Where
1	Controller position (headset's room tracking)	Measuring tip, 6 cm in front	Headset
2	Two touched points	The table frame (origin, directions) + LAMA size	Headset (calibration)
3	Laser ray	Point on the table, in the table frame	Headset (drawing)
4	Curve points	Brick plan (position, rotation, footprint)	Headset (wall logic)
5	Unity axes (y up, clockwise yaw)	Robot-style axes (z up, counter-clockwise yaw)	Headset (BrickSender)
6	Objects	JSON text → bytes in a UDP packet	Headset → laptop
7	Table frame, metres	Robot base frame, millimetres	Laptop (bridge)
8	Brick centre	Gripper tip target → flange target	Laptop
9	Flange target	6 joint angles (inverse kinematics)	Laptop
10	Radians (inside the maths)	Degrees (in the message)	Laptop
11	Joint angles	3D model part rotations (+ zero offsets)	Headset (ghost)
12	Robot targets	URScript text (`movej`, `movel`, gripper output)	Laptop → robot (future)
13	Rhino model (mm, z up)	STL → Unity mesh (m, y up)	Laptop (tray import)

6. Every file type in the project

Extension	What it is	Examples in our project
`.cs`	C# code (Unity scripts)	`WallBuilder.cs`, `TableCalibration.cs`, `GhostRobot.cs`, `GhostPlayer.cs`, `BrickSender.cs`
`.py`	Python code	`picobricks_bridge.py`, `ur5_kinematics.py`, `make_tray.py` (runs inside Rhino)
`.bat`	Windows batch file: a list of commands you double-click	`PICOBRICKS.bat` (the bridge's menu)
`.unity`	A Unity scene	`Assets/Scenes/SampleScene.unity`
`.prefab`	A reusable Unity object	`LAMA_UR5.prefab` (robot + table from HANDZONe)
`.mat`	A Unity material (colour, transparency)	`BrickTransparent.mat`, `GhostRobot.mat`, `DrawArea.mat`
`.asset`	A Unity data file (here: meshes, settings)	`PickupTray.asset`, `BrickOutline.asset`
`.meta`	Unity's ID card for every file, so references survive renames. Always move together with the file.	one next to every asset
`.fbx`	A common 3D model format (with parts and hierarchy)	HANDZONe's UR5, gripper, table
`.stl`	A simple 3D model format (triangles only), the standard for 3D printing	`pickup_tray_bricks_1-6.stl`, `test_fit_0.5.stl`
`.3dm`	Rhino's own model file	`pickup_tray.3dm`
`.json`	Structured text data	the messages, `wall_plan.json`, `last_layout.json`, `calib.json`, `LAMA_UR5_joints.json`, `Packages/manifest.json` (list of Unity packages)
`.csv`	A table as text, values separated by commas (opens in Excel)	`scene_objects.csv` (Matthias's exporter)
`.apk`	An installable Android app	`PicoBricks.apk` (not on GitHub)
`.keystore`	The private signing key for the app	`user.keystore` (never on GitHub; shared privately)
`.md`	Markdown: plain text with simple formatting, shown nicely on GitHub	`README.md`, `docs/DEVLOG.md`

`.script`	URScript program for the robot	`last_program.script` (written by the bridge)
`.gitignore`	List of files Git should ignore	in the project root
`.png .jpeg .mp4`	Images and videos	headset screenshots and screen recordings
`.pptx .pdf`	Slides, documents	catch-up slides, this document

7. Where everything lives

The GitHub repository (`jintecture/PicoBricks`, private)

Folder / file	What's in it
`README.md`	The front page: what the app does, the controls, the file map, how to start.
`docs/DEVLOG.md`	The development diary: every step, problem and decision, with dates.
`docs/HANDZONE_NOTES.md`	What we took from HANDZONe and how.
`docs/images/`	Pictures used in the docs.
`docs/explainer/`	This document.
`Assets/Scripts/`	The app's behaviour (C#).
`Assets/Editor/`	Tools that only run inside the Unity Editor: the scene setup and the tray importer.
`Assets/Scenes/`	The scene.
`Assets/Materials/`, `Assets/Models/`	Looks and shapes.
`Assets/ThirdParty/HANDZONe/`	The UR5, gripper and LAMA table models (40 MB).
`robot-bridge/`	The laptop side: bridge, UR5 maths, menu, calibration files, its own README.
`hardware/pickup-tray/`	The tray: generator script, STL and Rhino files, previews, README with all measurements.
`Packages/`, `ProjectSettings/`	Unity's package list and settings.
Not on GitHub (on purpose)	`Library/`, `Temp/`, `Logs/` (Unity regenerates them), the APK, the keystore, per-session files like `last_layout.json`.

Bookmarks and branches

- Tags: `v1-manual-placement` (15 Oct, before the wall-drawing version), `before-robot-link` (18 Oct, before the big merge).
- Branches: `snap-grid` (Parjanya, merged), `wall-builder` (Jin, merged), `robot-link` (Parjanya, merged), `tcp-receiver` (Matthias, not merged).

Other places

- The project on the laptop: `C:\Users\Jin\PicoBricks`.

- On the headset: the app, `wall_plan.json` in the app's folder, screenshots in `Pictures/Screenshots`, recordings in `Movies/ScreenRecording`.
- Google Drive: the Obsidian vault note "AR Brick Project" and copies of the headset captures ("Pico captures").
- The keystore: on Jin's laptop only; shared privately with teammates, never online.

8. Problems we hit, and what they teach

These make good stories when you explain the project, and each one teaches a general lesson.

Problem	Real cause	Fix	Lesson
No Android option in Unity	Incomplete install	Reinstall with Android modules	Install exactly the versions and modules a project needs.
Passthrough black (1)	Headset firmware older than the SDK expected	Use the older SDK (2.5.0) for the old headset	Software and hardware versions must match.
Passthrough black (2)	The HDR colour format had no transparency channel	HDR off during passthrough	"Transparent" needs an alpha channel all the way through.
Table above the head	Old firmware shifted tracking height by 1.15 m	Measure the real table instead of trusting the headset's floor	Measure the real world; don't assume.
Table sinking 1 m per calibration	The ready-made rig had gravity and walking	Switched off	Templates come with assumptions (games ≠ AR).
Rhino booleans failing	Inside-out cutting shapes	Flip, merge, subtract once	Check geometry direction ("normals").
Bricks drifting off S-curves	A small shift carried from brick to brick	Aim each brick from where it really starts	Correct errors locally so they don't add up. (Found by Jin from a drawing.)
Parjanya's script didn't compile	Two toolkits used the same names	Use the full names	Always compile after merging.
RoboDK "Invalid item"	RoboDK restarted, old handles dead	Reconnect before each run	Long-running programs must handle restarts.
RoboDK showing an older wall	Waiting for Y/n, and duplicate filter too broad	Smarter filter, newest wall wins (Y/n kept on purpose)	Check the data at each step before blaming the maths.
Ghost arm pointing the wrong way	Model's zero pose ≠ real robot's zero pose	+90° offsets, checked with known poses	Verify conventions with known test cases.
Unity build blocked by Windows	Smart App Control blocked an unsigned Unity file	Turned it off (Jin)	Security tools can block development tools.
RoboDK refusing commands	Trial licence expired	Own UR5 maths (Jin's decision)	Avoid depending on licences for core features when a known formula exists.
Tray inside the robot	Robot measured from the middle, tray from the left corner	Measure both from the same point	Keep things that belong together in one reference frame.

Walls refused although the area was enlarged	The old value stayed stored in the scene	Setup clears it explicitly	Removing a setting isn't the same as resetting it.
Bricks overlapping on sharp curves	Hinge corner guessed, result not checked	Try both, check against all bricks	Check the result, not just the rule.
Bricks floating and tilted	A button read by two scripts; A during the preview resent half-moved bricks	Preview + one clear confirm step	One button, one meaning at a time.

9. The rest of the team and the two robot routes

Suzanne: computer vision for taking the wall apart

A Raspberry Pi 3 with a camera on a tripod films the table and streams it to the laptop (RTSP stream via "mediamtx"). ArUco markers (square black-and-white codes) lie flat on the table and are stuck on the bricks. Her "Table Camera Toolkit" (Python) calibrates the camera with a checkerboard, finds the markers, and works out the table plane and each marker's height. Grasshopper turns that into brick positions, and the Robots plugin makes the UR5 remove the top layer; then the table is scanned again, until the wall is gone.

Two routes from the headset to the robot (not decided yet)

	Route A: Grasshopper (Matthias)	**Route B: Python bridge (Parjanya + Jin)**
Chain	Unity → live TCP link → Grasshopper "TCP Receiver" → Robots plugin → UR5	Headset → Wi-Fi → Python bridge (own UR5 maths) → ghost back in AR → UR5
Data	Brick ID, XYZ, yaw	Pick + place per brick, plus the robot preview
Status	Unity → Grasshopper connection shown working	Ghost robot builds the wall in AR; not yet run on the real robot
Strengths	Same tool as the CV disassembly; robot path visible in Rhino; the team knows Grasshopper	Preview in AR before the robot moves; no licence needed; the ghost and the robot get the same joint angles

A possible combination: Grasshopper drives the robot (shared with the computer vision part), and the headset still shows the preview. Somebody would then have to make Grasshopper send the robot's movements back to the headset.

10. Open questions, next steps, and how to explain it yourself

Next steps

1. **A lab session with Matthias:** print and tape the tray beside the robot; measure the two table corners with the robot (menu 5) and check (menu 6); find out the real gripper length, which output opens and closes it, and whether it can grip a brick lying flat from the top.
2. Check that the ghost robot stands exactly on the real robot in the headset.
3. Send the robot exactly the joint angles the ghost showed.
4. Triangle walls (3 + 2 + 1 bricks).

Team decisions

- Robot route: A, B, or a combination.
- One shared brick list format (the headset already sends JSON).
- Stay on the Pico or move to a Meta Quest (HANDZONe uses a Quest). Estimated about a day of work, mostly passthrough.

The project in 1 minute

> "We're making a robot a design partner instead of a labourer. You put on a headset and see the real table, with a robot arm and six timber bricks. You draw a curve on the table with the controller, choose how many layers, and the bricks immediately form that wall in AR. When you press A, the wall goes over Wi-Fi to a laptop, which calculates every movement the robot arm needs and sends it back. A see-through robot in the headset then builds your wall brick by brick, exactly as the real robot will. Next, the real UR5 builds it, and a camera system lets the robot take it apart again for the next visitor."

The project in 5 minutes (talking points)

1. **Concept:** human-led design, the robot as a creative partner; the user must see and confirm what the robot will do (communication between human and robot).
2. **AR, not VR:** passthrough shows the real room; virtual things are lined up with the real table by touching two points.
3. **Designing:** draw a curve, choose 1–3 layers; the app turns the curve into brick positions with two simple rules (dividers and folding ruler), keeps it within the robot's reach, and checks bricks never overlap.
4. **Confirming:** the bricks preview the wall; A sends it.
5. **Data:** a short JSON message over Wi-Fi to the laptop; the laptop converts it from table coordinates to robot coordinates and solves the robot's joint angles (inverse kinematics), about 200 poses.

6. **Rehearsal:** the poses go back to the headset, where a ghost UR5 acts out the build.
7. **Real robot:** after a second confirmation on the laptop, the same movements go to the UR5. The 3D-printed tray makes sure every real brick starts in a known spot.
8. **Team:** Suzanne's camera system for disassembly, Matthias's Grasshopper route; the routes still have to be combined or chosen.

Questions people might ask you

Question	Short answer
How does the headset know where the table is?	It doesn't, until you touch two points on the table's front edge. From those and the known table size it places everything.
How does the robot know where the bricks are?	They start in a 3D-printed tray at a fixed spot; the same tray is in the app, so every pocket's position is known.
What exactly is sent between devices?	Short JSON text messages over Wi-Fi: the wall (brick pick and place positions) one way, the robot's joint angles the other way.
Why not just use RoboDK?	Its licence expired, and the robot's joint maths is a known exact formula; doing it ourselves needs no licence and the preview uses exactly the robot's angles.
Why does the wall sometimes move a bit?	If it pokes out of the robot's reach, the app slides it to the nearest spot that fits, keeping the shape.
Why only 6 bricks and 3 options?	A deliberate first version (rectangle). It gets the whole chain working; a triangle pattern is next.
Is it safe?	Two confirmations (headset and laptop), slow speeds for first tests, emergency stop in hand, and the ghost preview shows every move first.
What did you learn?	Version control with Git, Unity and AR, coordinate systems, how robots are controlled (forward and inverse kinematics), and a lot of debugging: measure, check each step, change one thing at a time.

Sources: the project's GitHub repository (README, DEVLOG, HANDZONE notes, robot-bridge and pickup-tray READMEs, code), the group's catch-up slides (ShortPitchHRC), and the working sessions of 9–21 October 2026.